Listing 6.5 Splitting a text into a list of chunks for retrieval

```
const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 100,
  chunkOverlap: 20,
});
const output = await splitter.createDocuments([text]);

/* Example output structure:
[
  Document {
    pageContent: "Artificial Intelligence (AI) is intelligence",
    metadata: { id: 2 }
  },
  Document {
    ...
  },
  ...
]
*/
```

Sets the size of each text chunk to 100 characters

Defines a 20-character overlap between chunks to maintain context continuity

Splits the input text into a list of Document objects based on the configuration

The `RecursiveCharacterTextSplitter` takes two key parameters. First, `chunkSize` defines the maximum number of characters in each chunk. Second, `chunkOverlap` specifies how many characters should overlap between chunks to maintain context. The `createDocuments` method processes the text and returns a list of `Document` instances. Each `Document` contains a chunk of text and its metadata, making it ready for tasks like generating embeddings or storing in a vector database.

Once you have prepared the list of documents, the next step is to ingest them into a database optimized for storing embeddings. This task is handled by vector stores.

6.2.2 Introducing vector stores

Vector stores are specialized databases designed to store and efficiently query high-dimensional vector data. In the context of natural language processing and AI applications, these vectors typically represent embeddings of text documents or chunks of text.

In the previous chapter, we learned that embeddings are floating-point numbers that represent an encoded value of a prompt that can be used in a specific model. With these numbers, we can perform various operations, such as similarity searches, clustering, and classification. Vector stores provide several advantages here. They contribute to efficient similarity search by allowing for quick retrieval of documents that are similar to a given query vector; they can handle large numbers of vectors and perform fast searches even as the dataset grows; and they can seamlessly integrate with machine learning models, allowing for real-time updates and retrievals based on model output.

Since we are working with LangChain, we can use the various adapters that interface with open source vector databases like Supabase, Pinecone, or Chroma. The official